

Guru Nanak College of Arts, Science and Commerce

G.T.B. Nagar, Mumbai – 400037

(Autonomous)

NAAC Accredited 'A+'

DEPARTMENT OF INFORMATION TECHNOLOGY



MSC (IT) PART II SEMESTER III

PROJECT OF

Plant Disease Classification Using Convolutional Neural Networks

SUBMITTED

BY

GUPTA KETKI RAJESH

Seat No.: 24-25/101

ACADEMIC YEAR 2024-2025

UNDER THE GUIDANCE OF

HARPREET KAUR

Guru Nanak College of Arts, Science and Commerce

G.T.B. Nagar, Mumbai – 400037

(Autonomous)

NAAC Accredited 'A+'

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project entitled, **Plant Disease Classification Using Convolutional Neural Networks** is Bonafide work of **KETKI GUPTA** bearing Seat. No: **24-25/101** submitted in partial fulfilment of the requirements for the award of degree of MASTER OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Examiner

Coordinator

External Examiner

Abstract

This document details the development and evaluation of a deep learning model for the automated detection of plant diseases using the publicly available PlantVillage dataset. The model, trained with cross-entropy loss and optimized using the fit_one_cycle learning rate policy on a GPU, achieved a high test accuracy of 99.2% in classifying 38 distinct categories of healthy and diseased plant leaves. The training process, including the evolution of accuracy and loss over epochs, is analyzed to provide context for the final performance. While the high accuracy demonstrates the model's potential for real-world application in agricultural settings, the training dynamics suggest areas for future improvement, such as addressing potential overfitting and class imbalance. Proposals for future work include evaluating the model on more diverse datasets, enhancing its robustness and interpretability through advanced techniques, and exploring deployment strategies for practical use. Supplementary details regarding the experimental setup, detailed evaluation metrics, and training configurations are provided in the appendices.

Table of Contents

Chapter No	Title	Page no
	Abstract	1
1	Introduction	3
2	Literature Review	4
3	Scope of project	6
4	Objective of project	7
5	Technologies to be used	8
6	Methodology	9
7	Deployment	23
8	Results	25
9	Conclusion and Proposals of future work	30
	Bibliography	32
	Appendices	33
	• Screenshots	

List of Figures

Fig no	List of Figure
1	Block Diagram of Proposed System
2	Images of leaves
3	Sample images of color, grayscale and segmented leaves
4	Architecture of CNN
5	Residual Block
6	ResNet-9 Architecture
7	Residual Connections in ResNet-9
8	Model Training
9	Workflow of project
10	Accuracy vs. Number of Epochs
11	Loss vs. Number of Epochs
12	Learning Rate vs Batch Number
13	Test Images
14	Prediction screenshots

List of Tables

Table no	List of Figure
1	Detailed description of PlantVillage dataset with relative information.
2	ResNet Types
3	Detailed Layer-by-Layer Architecture
4	Summary for Streamlit Deployment

Introduction

Crop production can be greatly reduced due to various diseases, which seriously endangers food security. Thus, detecting plant diseases accurately is necessary and urgent. Traditional classification methods, such as naked-eye observation and laboratory tests, have many limitations, such as being time consuming and subjective. Currently, deep learning (DL) methods, especially those based on convolutional neural network (CNN), have gained widespread application in plant disease classification. They have solved or partially solved the problems of traditional classification methods and represent state-of-the-art technology in this field. In this work, we reviewed the latest CNN networks pertinent to plant leaf disease classification.

The Food and Agriculture Organization of the United Nations, accessed on 5 December 2020) reported that the number of hungry people in the world has been increasing slowly since 2014. Current estimates show that nearly 690 million people are hungry, and they account for 8.9% of the world's total population; this figure represents an increase of 10 million in 1 year and nearly 60 million in 5 years. Meanwhile, more than 90% of people in the world rely on agriculture. Farmers produce 80% of the world's food; however, more than 50% of crop production is lost due to plant diseases and pests. Thus, recognizing and detecting plant disease accurately is necessary and urgent.

Agriculture is a fundamental industry that ensures global food security, but it faces significant challenges due to plant diseases. Early detection of plant diseases is crucial to mitigate crop losses and improve yield. Traditionally, plant disease diagnosis has been done manually by experts, which is time-consuming and often prone to human error. With the advent of deep learning, particularly Convolutional Neural Networks (CNNs), there is an opportunity to automate the classification of plant diseases using image data.

Early detection of plant disease aids in the prevention of large-scale crop losses. Farmers must apply the appropriate insecticides for their crops. Too many pesticides are harmful to crops and farmland. Getting expert advice will help you avoid misusing chemicals on plants. Plants have been the focus of many researchers to aid farmers and others involved in agriculture. When a disease is visible to the naked eye, it is straightforward to detect. The illness may be discovered and treated early if the farmer has sufficient information and monitors the crops on a regular basis. However, this phase only exists when the disease is extreme or crop output is low. Then there are the different innovations. Farmers will benefit from the introduction of automated disease detection tools. This approach yields outcomes that are suitable for both little and large-scale agricultural cultivation.

Importantly, the results are precise, and the disorders are detected in a very short amount of time. These technologies rely heavily on deep learning and neural networks to function. Deep Convolutional Neural Network is utilized in this study to identify infected and healthy leaves, as well as to detect illness in afflicted plants. The CNN model is designed to suit both healthy and sick leaves; photos are used to train the model, and the output is determined by the input leaf. This project aims to develop a robust model for classifying plant diseases using the PlantVillage dataset, which contains a diverse collection of images of healthy and diseased plant leaves. By employing pretrained models and CNN architectures, the project seeks to enhance the accuracy and efficiency of disease detection, ultimately aiding farmers and agricultural professionals in managing crop health.

Literature Review

Plant pathologists analyze different parts of the plants like root, kernel, stem and leaf for the classification of plant diseases. Various studies have explored the use of machine learning algorithms for plant disease detection. Traditional image classification techniques, such as SVM, decision trees, and KNN, have been employed but often lack the accuracy and scalability required for large datasets. Convolutional Neural Networks (CNNs) have shown superior performance in image classification tasks due to their ability to automatically learn hierarchical features from raw data.

Mohanty et al. (2016) and Ferentinos (2018) have successfully applied CNNs for plant disease classification using datasets like PlantVillage. Hughes, D. P., & Salathé, M, the paper introduces the PlantVillage dataset, which has become a standard dataset for plant disease classification tasks and forms the basis for many machine learning applications in agriculture.

"Going deeper with convolutions" *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* The paper presents the Inception network, which is widely used in image classification tasks and has been applied to plant disease detection.

For instance, researchers have successfully applied CNN architectures to classify diseases in crops such as rice, corn, and wheat. The use of pretrained models, such as VGG16, ResNet, and Inception, has further enhanced classification accuracy by leveraging features learned from large datasets. Szegedy, C., et al. (2015). Additionally, studies have highlighted the importance of data augmentation techniques in improving model robustness and generalization.

The study of plant leaf disease detection by Malvika Ranjan and colleagues starts with image capturing. Color data, such as HSV features, are retrieved from the segmentation results, and an artificial neural network (ANN) is then trained by selecting feature values that can effectively discriminate between healthy and sick samples. Using a combination of image data processing methods and ANN, the current study suggests a method for identifying cotton leaf illnesses early and reliably.

Srdjan Sladojevic and colleagues present Deep Convolutional Neural network Supported Identification of Crop Diseases by Plant Image Classification, a new method for the construction of a crop diseases recognition model based on plant image classification and deep convolutional networks. The methodology employed and the novel technique of training allow for a quick and painless system set up in practice. With the ability to identify crops from their surroundings, the built model can recognize thirteen types of plant illnesses from healthy leaves. All of the necessary processes for applying this diseases recognition model are detailed throughout the study, beginning with the collection of photographs in order to establish a database that is evaluated by agricultural experts. Caffe, a deep learning framework developed by Berkley Vision and Learning Centre, was used to perform the deep CNN training. The experimental results on the developed model achieved precision between 91% and 98%, for separate class tests, on average 96.3%.

CNN and Res Net were used to classify plant diseases. Others, like Emanuel Cortes A deep neural network and semi-supervised algorithms were trained to distinguish crop species and disease status of 57 different classes using a publicly available dataset of 86,147 photos of ill and healthy plants. rs-net was the unlabeled data experiment that functioned successfully. With a detection rate of $1e-5$, it was able to score more than 80% in the training phase in less than 5 epochs.

Plant disease identification and treatment using neural network models, Konstantinos P. Ferentinos and colleagues built CNN models to conduct crop disease identification and diagnosis using basic leaf pictures of healthy and sick plants. The models were trained using an open collection of 87,848 photos, which included 25 kinds of plants in 58 various classes of [plant, illness] pairs, including non-affected plants. Multiple model architectures were developed, with the topperforming one achieving a success rate of 99.53 percent. The model's high success rate makes it a valuable or early detection tool.

The PlantVillage collection yielded of about 87K rgb images of healthy and diseased crop leaves which is categorized into 38 different classes. The photos were taken in an unstructured setting using offline augmentation. The built model obtains an accuracy of classification of 99.2 percent, demonstrating it a Convolutional neural network effectively extract significant features and diagnose plant diseases from photos captured in the wild.

The proposed model was trained for small object detection and the model performed better than the other state-of-the-art object detection models. Bhattacharya et al. proposed an approach to classify bacterial blight, rice blast and brown mark disease images (total of 1500 images) with 94% accuracy. In this paper, we have proposed deep learning-based approach whose hyper-parameters can be optimized by using nature-inspired optimization algorithms.

The convolutional neural network was used by Garima Shrestha and colleagues to identify plant disease in 2020. With an accuracy of 88.80 percent, the authors were able to classify 12 plant diseases. Experimentation was carried out by using a collection of 3000 high-resolution RGB photographs. The convolutional layer and pooling layer have 3 blocks in this network. Eventually, the network becomes very expensive as a result of this. Additionally, the model's F1 score is 0.12, which is extremely poor due to the significant amount of erroneous negative predictions.

A Deep-Learning-Based Detection for static of “Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Pepper, Potato, Raspberry, soyabean, Strawberry, Tomato” and Diseases are “Squash Powdery Mildew, Cedar Apple Rust, Powdery Mildew, Cercospora Leaf Spot, Common Rust, Northern Leaf Blight, Black Rot, Esca Black Measles, Isariopsis Leaf Spot, Haunglongbing Citrus Greening, Bacterial Spot, Early Blight, Late Blight, Leaf Scorch, Leaf Mold, Septoria Leaf Spot, Spider mites two spotted spider mite, Mosaic Virus and Yellow Leaf Curl Virus” and model is based on Convolutional Neural Network, all of which are referred to as "deep learning meta-architectures" in this paper. We use "deep feature extractors" like Residual Network to merge every one of these meta-architectures (ResNet). We show how deep morpho and feature extractors perform, and we also suggest a way for locally and globally category labeling and feature extraction to improve accuracy and reduce false positives throughout training. We train and test our systems end-to-end on our large PlantVillage Dataset and Pests Dataset, which contains challenging images of diseases and pests, including several inter-and extra-class variations, such as infection status and location in the plant.

Scope of the Project

The project titled "**Plant Disease Detection Using Convolutional Neural Networks (CNN)**" focuses on building an intelligent, automated system that can accurately identify and classify plant diseases based on visual symptoms observed on the leaves. This project aims to support farmers, agricultural scientists, and researchers by providing a fast, efficient, and accessible method to detect diseases early, thereby reducing the risk of widespread crop damage and improving agricultural productivity. **Data Collection:** The primary dataset used will be the PlantVillage dataset, which contains thousands of labeled images of plant leaves affected by various diseases.

- **Data Collection and Preparation:** Collection of plant leaf images from publicly available datasets such as PlantVillage or through self-captured images. Applying techniques like resizing, normalization, denoising, and enhancement to prepare the images for CNN input. Techniques like rotation, flipping, zooming, and cropping to artificially expand the training dataset and improve model robustness.
- **Model Development:** Designing a Convolutional Neural Network suitable for image classification tasks. The architecture may include convolutional layers, pooling layers, dropout layers, and fully connected layers. Exploring the use of pre-trained models like ResNet, VGG16, or MobileNet to leverage existing feature extraction capabilities. Fine-tuning parameters such as learning rate, batch size, number of epochs, activation functions, and optimizer choices to improve performance.
- **Model Training and Evaluation:** Training the CNN model on the prepared dataset using GPU acceleration for faster computation. Measuring model accuracy, precision, recall, F1-score, and confusion matrix on validation and test datasets. Applying cross-validation techniques to ensure model reliability and avoid overfitting.
- **Deployment and Usability:** Developing a simple interface (web or mobile-based) where users can upload images and receive immediate predictions. Implementing live image capture through webcams or mobile cameras for on-field usage. Displaying model predictions, probabilities.
- **Limitations:** The system is limited to detecting diseases that have visible symptoms on leaves. Diseases affecting internal plant parts or soil conditions are beyond the project's scope. The model's effectiveness will depend on the number and diversity of plant species and disease types included during training. The system may be sensitive to image capture conditions like lighting, background noise, and camera quality, which may slightly affect accuracy.
- **Future Scope:** Increasing the number of plant species and diseases handled by the model. Designing interfaces in multiple languages for better accessibility to farmers worldwide. Linking with drones or smart farming sensors for automated large-scale monitoring. Providing users with treatment or preventive suggestions based on detected diseases in future versions.

Objective of the Project

The main objective of this project is to develop a deep learning-based plant disease classification system. Specific objectives include:

- Collecting and preprocessing the PlantVillage dataset for model training.
- To develop a convolutional neural network model for the classification of plant diseases using images of leaves.
- To evaluate the model's performance using metrics such as accuracy, precision, recall, and F1-score.
- To implement data augmentation techniques to improve model robustness.
- Developing a simple interface for users to upload plant images and receive disease classification predictions.

Technologies to be Used

Hardware Technologies:

- Computer System: A machine with a dedicated GPU (e.g., NVIDIA GTX/RTX series) for efficient training of deep learning models.
- Camera/Smartphone: For capturing images of plant leaves for testing and validation.
- Storage: Local or cloud-based storage will be required for handling large image datasets like PlantVillage.

Software Technologies:

- Operating System: Windows.
- Integrated Development Environment (IDE): Jupyter Notebook or PyCharm or Google Colab for coding and experimentation.
- Version Control: Git for source code management and collaboration.
- Deep Learning Frameworks:
- TensorFlow/Keras: For building and training the CNN model and utilizing pretrained models.
- Data Processing Libraries:
- NumPy: For numerical operations and array manipulation.
- Pandas: For data handling and preprocessing tasks.
- OpenCV: For image processing tasks like resizing and augmentation.
- Matplotlib/Seaborn: For data visualization and presenting training results.
- Transfer Learning Models: Pretrained models such as VGG16, ResNet, and Inception for transfer learning.

Methodology

Proposed System:

We are building a neural network model for image classification. this model will be deployed on the web application for static detection of plant leaf disease through a file uploading system. The recognition and classification procedures are depicted in Fig. 1

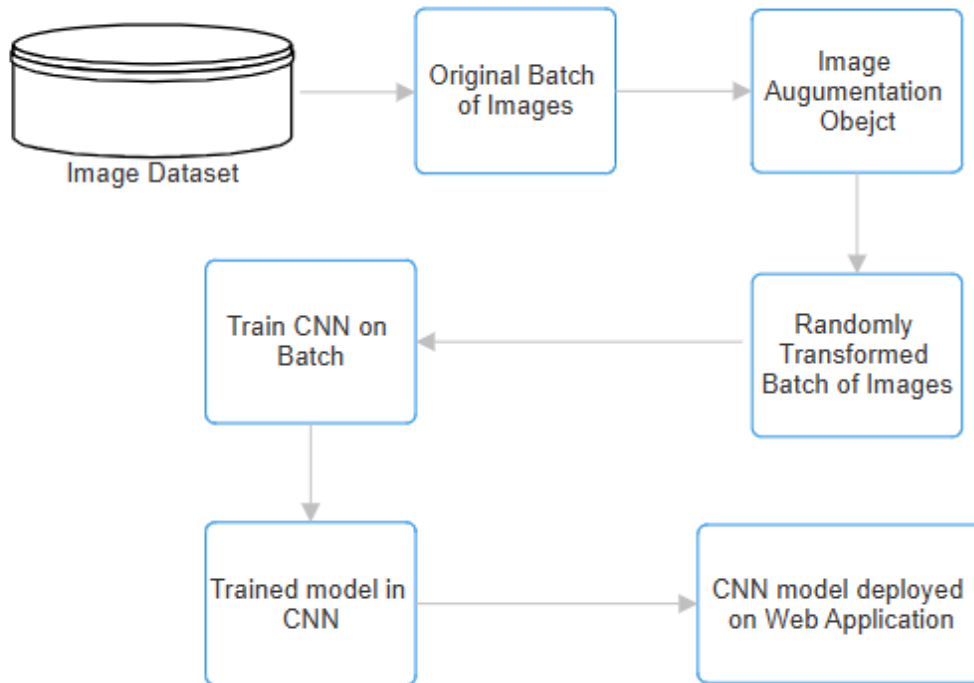


Fig. 1. Block Diagram of Proposed System

- The first step is to collect data. We are using the PlantVillage Dataset, which is widely available. This dataset was released by PlantVillage.
- Pre-processing and Augmentation of the collected dataset is done using pre-processing, image detection through CNN and description given Gemini model.
- Building CNN (Convolutional Neural Network) Model (Res-Net architecture) for classification of various plant diseases.
- Developed model will be deployed on the Web Application with help of Streamlite.

Dataset

Dataset Source: In this project, the PlantVillage dataset is used for training and evaluating the Convolutional Neural Network (CNN) model. The PlantVillage dataset was released by researchers from Pennsylvania State University under the leadership of Dr. David Hughes. It is an open-access dataset designed to support the development of machine learning tools for plant disease diagnosis. The dataset was developed to assist in the application of machine learning techniques for plant disease diagnosis and is publicly available through various platforms including Kaggle ([PlantVillageDatasetLink](#)).

Description of Dataset:

The dataset contains approximately 54,000 images of plant leaves, covering over 38 different classes including both healthy leaves and leaves affected by various diseases. The images span 14 crop species are tomatoes, potatoes, grapes, apples, corn, blueberry, raspberry, soybeans, squash and strawberry. Images are provided in high-resolution **JPEG** format

			
Apple Scab	Apple healthy	Blueberry Healthy	Cherry Powdery mildew

			
Corn Common Rust	Grape Black Rot	Haunglongbing Citrus	Peach Bacterial Spot

Fig 2. Images of leaves

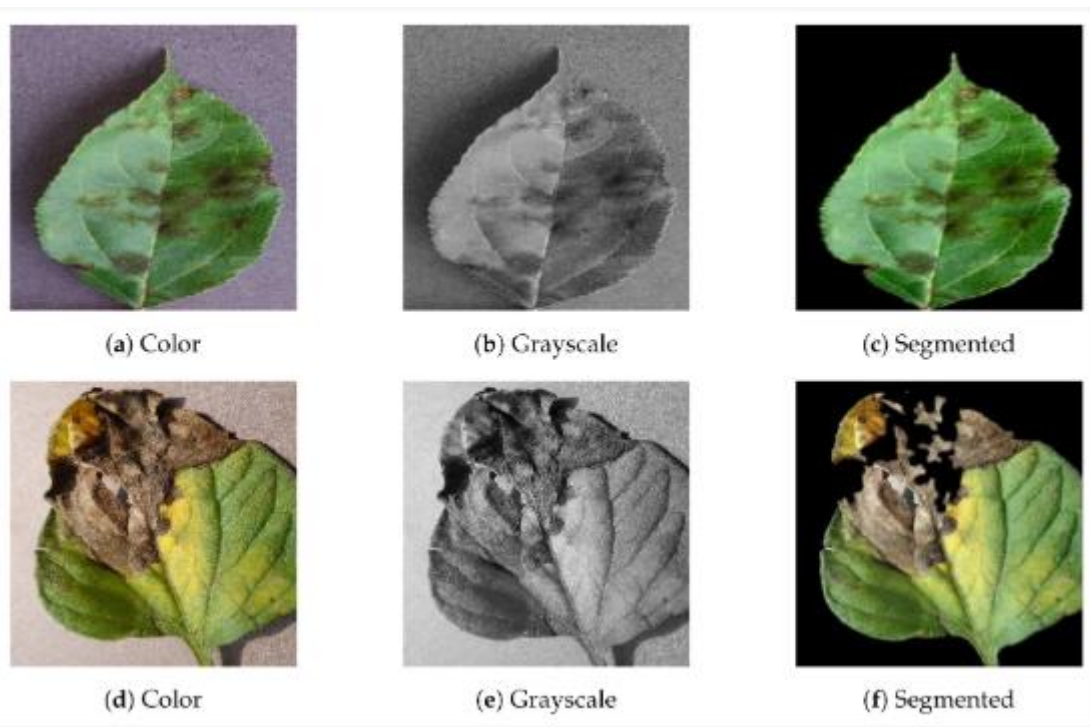


Fig 3. Sample images of color, grayscale and segmented version of PlantVillage image dataset.

Table 1 Detailed description of PlantVillage dataset with relative information.

Class	Plant Name	Disease Name	Causes Virus Name	Type of Disease	No. of Images
C1	Apple	Healthy	-	-	1645
C2	Apple	Apple scab	<i>Venturiainaequalis</i>	Fungus	630
C3	Apple	Black rot	<i>Botryosphaeriaobtusa</i>	Fungus	621
C4	Apple	Cedar apple rust	<i>Gymnosp-orangium</i>	Fungus	275
C5	Blueberry	Healthy	-	-	1502
C6	Cherry	Healthy	-	-	854
C7	Cherry	Powdery mildew	<i>Podosphaeraclandestina</i>	Biotrophic Fungus	1052
C8	Corn	Healthy	-	-	1162
C9	Corn	Cercospora leaf spot	<i>Cercosporazeae-maydis</i>	Fungal	513
C10	Corn	Common rust	<i>Puccinia sorghi</i>	Fungus	1192
C11	Corn	Northern Leaf Blight	<i>Exserohilumturcicum</i>	Foliar	985
C12	Grape	Healthy			423
C13	Grape	Black rot	<i>Guignardiabidwellii</i>	Fungus	1180
C14	Grape	Esca (Black Measles)	<i>Phaeomoniellachlamydospora</i>	Fungus	1383
C15	Grape	Leaf blight (Isariopsis)	<i>Pseudocercosporavitis</i>	Fungus	1076

C16	Orange	Healthy	-	-	5507
C17	Peach	Healthy	-	-	360
C18	Peach	Bacterial spot	<i>Xanthomonas campestris</i> pv. Pruni	Bacterial	2297
C19	Pepper/bell	Healthy	-	-	1478
C20	Pepper/bell	Bacterial spot	<i>Xanthomonas campestris</i> pv.	Bacterial	997
C21	Potato	Healthy	-	-	152
C22	Potato	Early blight	<i>Alternaria solani</i>	Fungal	1000
C23	Potato	Late blight	<i>Phytophthora infestans</i>	Fungal	1000
C24	Raspberry	Healthy	-	-	371
C25	Soybean	Healthy	-	-	5090
C26	Squash	Powdery mildew	<i>Podosphaera xanthii</i>	Fungal	1835
C27	Strawberry	Healthy	-	-	456
C28	Strawberry	Leaf scorch	<i>Diplocarpon earliana</i>	Fungal	1109
C29	Tomato	Healthy	-	-	1591
C30	Tomato	Bacterial spot	<i>Xanthomonas perforans</i>	Bacterial	2127
C31	Tomato	Early blight	<i>Alternaria</i> sp.	Fungal	1000
C31	Tomato	Late blight	<i>Phytophthora infestans</i>	Fungal	1909
C33	Tomato	Leaf Mold	<i>Lycopersicon</i>	Fungal	952
C34	Tomato	Septoria leaf spot	<i>Septoria lycopersici</i>	Fungal	1771
C35	Tomato	Spider mites	<i>Tetranychus</i> spp.	Pest	1676
C36	Tomato	Target Spot	<i>Corynespora cassicola</i>	Fungal	1404
C37	Tomato	Tomato mosaic virus	Tomato mosaic	Viral	373
C38	Tomato	Tomato Yellow Leaf	<i>Begomovirus</i>	Viral	5357

Data Preprocessing: Before feeding the images into the CNN model:

- Resizing: All images were resized to **224x224 pixels** to match the input size requirement of the ResNet architecture.
- Normalization: Pixel values were scaled between **0 and 1** to speed up training and improve convergence.

Data Augmentation: Techniques such as rotation, horizontal and vertical flipping, zooming, and brightness adjustments were applied to increase dataset variability and prevent model overfitting.

Data Splitting: In our experiment, we used three different formats of PlantVillage datasets. First, we ran the experiment with colored leaf images, and then with segmented leaf images of the same dataset. In the segmented images, the background was smoothed, so that it could provide more meaningful information that would be easier to analyze. Lastly, we used grayscale images of the same dataset to evaluate the performance of the implemented methods. All leaf images were divided into two sets, a training set and the testing set. To evaluate performance, we split leaf images into three different sets, namely 80–20 (80% training images and 20% testing images), 70–30 (70% training images and 30% testing images), and 60–40 (60% training images and 40% testing images).

Challenges in the Dataset:

- **Class Imbalance:** Some disease categories have significantly fewer images compared to others.
- **Visual Similarities:** Some diseases have very similar visual symptoms, making classification harder.
- **Environmental Variations:** Different lighting conditions and background noises may slightly affect the model's ability to generalize.

These challenges were addressed using data augmentation and careful CNN model design.

Convolutional Neural Network Architecture (ResNet)

Introduction to CNN: Convolutional Neural Networks (CNNs) are a specialized type of deep neural networks designed to process data with grid-like topology, such as images. CNNs automatically and adaptively learn spatial hierarchies of features through backpropagation by using multiple building blocks, such as convolution layers, pooling layers, and fully connected layers. In traditional CNNs, increasing the depth (i.e., number of layers) generally improves performance. However, very deep networks face problems like **vanishing gradients**, making them hard to train.

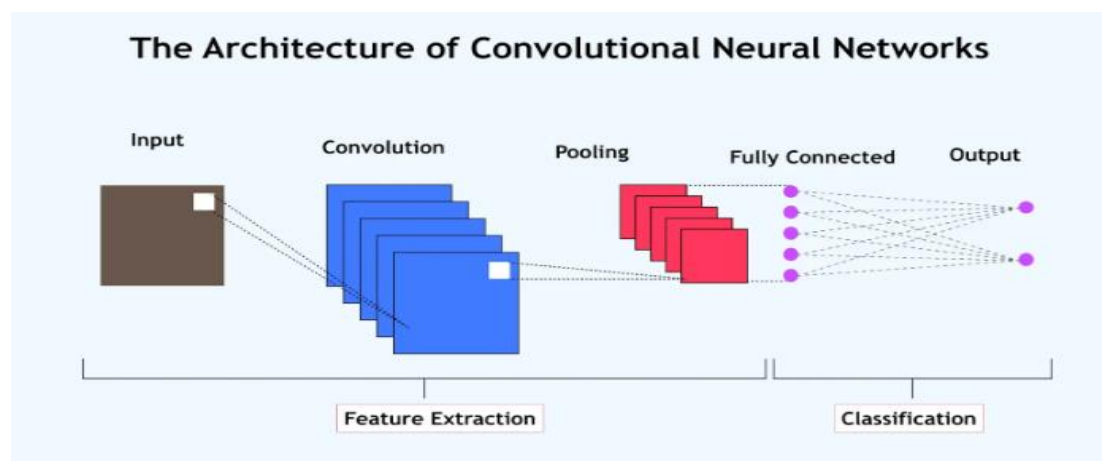


Fig 4. Architecture of CNN

Introduction to ResNet: ResNet (short for **Residual Network**) was introduced by **Kaiming He et al.** in their 2015 paper titled "*Deep Residual Learning for Image Recognition*". ResNet solved the training issues of very deep neural networks by introducing a novel concept called **residual learning**. Instead of learning the full output mapping directly, the network learns **residuals** — the difference between the input and output — making it much easier to optimize.

Key innovation: Skip Connections (also called shortcut connections).

Residual Block: A **residual block** is the basic building unit of a ResNet. Instead of directly mapping input x to output $H(x)$, the residual block makes the network learn $F(x) = H(x) - x$

$F(x) = H(x) - x$, $F(x) = H(x) - x$ (i.e., the residual function).

The final output is: $\text{Output} = F(x) + x$

Structure of a Residual Block:

Convolution $\rightarrow$ Batch Normalization $\rightarrow$ ReLU $\rightarrow$ Convolution $\rightarrow$ Batch Normalization.

Add the original input (**skip connection**) to the output. Apply ReLU activation again. Skip connections allow the gradient to flow directly through the network, preventing vanishing gradient issues.

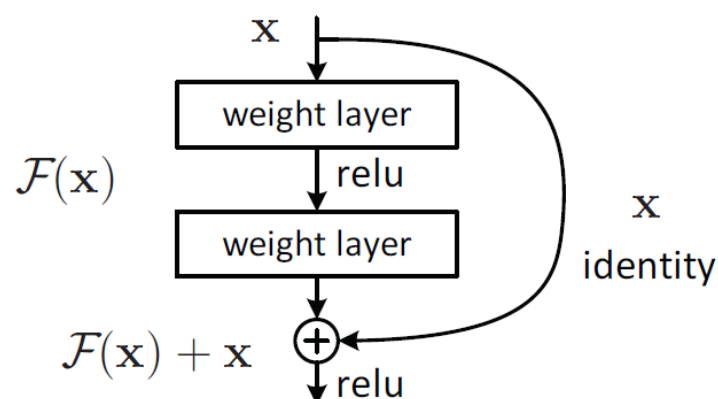


Fig 5. Residual Block

ResNet Variants: There are multiple versions of ResNet based on the number of layers as given in table below:

Table 2: ResNet Types

ResNet-9	9 layers
ResNet-18	18 layers
ResNet-34	34 layers
ResNet-50	50 layers (uses bottleneck blocks)

Bottleneck design (used in ResNet-50 and deeper models). Instead of two 3x3 convolutions, it uses, 1x1 Convolution $\rightarrow$ 3x3 Convolution $\rightarrow$ 1x1 Convolution. Reduces computation while maintaining performance.

For plant disease detection, **ResNet-18** or **ResNet-50** is often a good choice because they are deep enough but not too heavy to train on standard hardware.

ResNet-9 Architecture:

Introduction: ResNet-9 is a smaller and simpler version of the ResNet family designed mainly for small datasets or lightweight models where very deep networks like ResNet-50 are unnecessary.

It follows the same residual learning principle — using skip connections — but with fewer layers, making it much faster to train while still being effective for classification tasks like plant disease detection.

Structure of ResNet-9: ResNet-9 has, Convolutional layers for feature extraction. Residual (skip) connections to allow easier flow of gradients. Fully connected layer for classification. In total, it consists of around 9 weight layers: 8 convolutional layers + 1 fully connected layer.

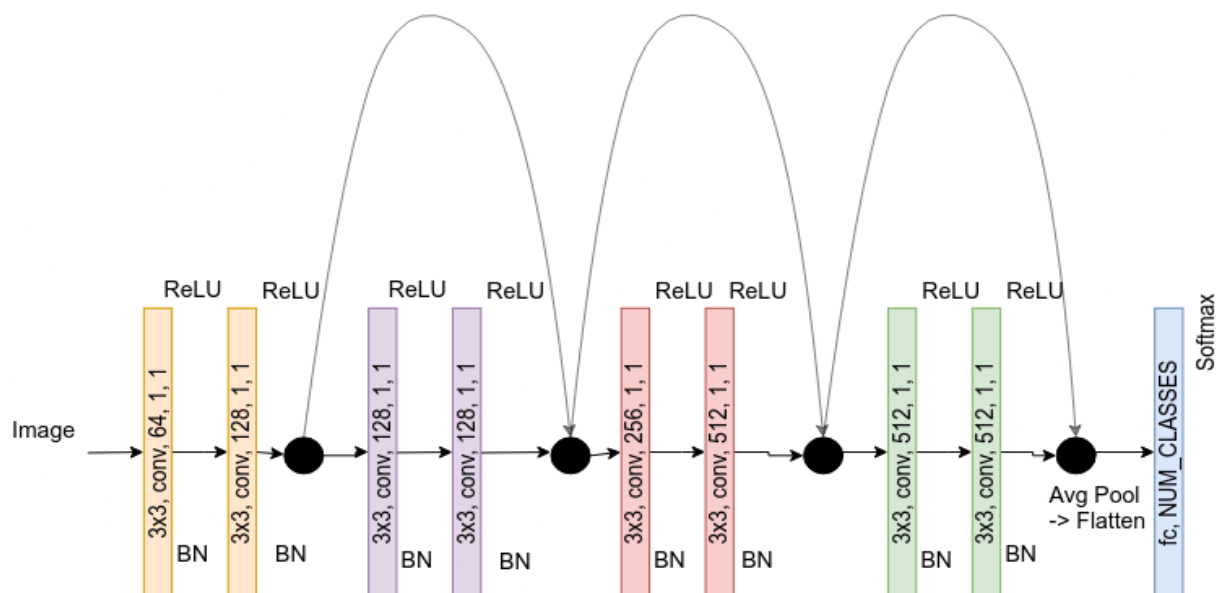


Fig 6. ResNet-9 Architecture

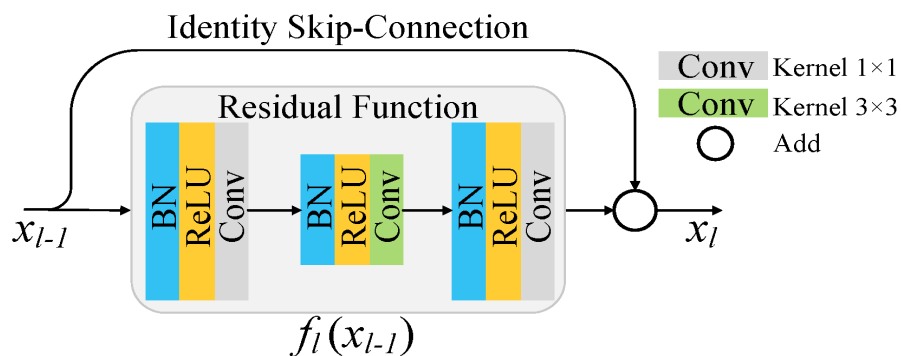
Table 3. Detailed Layer-by-Layer Architecture

Stage	Layer Type	Details
Input	Input Layer	Image (e.g., 224x224x3)
1	Conv Layer	3x3, 64 filters, stride 1, padding 1
2	BatchNorm + ReLU	Activation and normalization
3	Conv Layer	3x3, 128 filters, stride 2, padding 1
4	BatchNorm + ReLU	Activation and normalization
5	Residual Block	2 x (Conv 3x3, 128 filters) + Skip Connection
6	Conv Layer	3x3, 256 filters, stride 2, padding 1
7	BatchNorm + ReLU	Activation and normalization
8	Residual Block	2 x (Conv 3x3, 256 filters) + Skip Connection
9	Conv Layer	3x3, 512 filters, stride 2, padding 1
10	BatchNorm + ReLU	Activation and normalization
11	Global Average Pooling	Reduces feature map to 1x1
12	Fully Connected Layer	Output Layer (e.g., 38 classes for diseases)
13	Softmax Activation	For multi-class probability output

Visual flow of ResNet-9 (Mathematical form):

Input Image → Conv + BN + ReLU → Conv + BN + ReLU → Residual Block → Conv + BN + ReLU → Residual Block → Conv + BN + ReLU → Global Avg Pool → Fully Connected Layer → Softmax Output

Residual Connections in ResNet-9: There are two residual blocks: After the second convolution (with 128 filters), After the third convolution (with 256 filters). In each block, the input is added back to the output of two convolutional layers, making it easier to train.

**Fig 7.** Residual Connections in ResNet-9

Characteristics: Fewer parameters than deeper ResNets → faster training, less memory usage. Good accuracy on small-to-medium datasets like PlantVillage. Skip connections help avoid overfitting and vanishing gradients. Works well even without very deep layers thanks to residual learning.

Gemini (Prompt Engineering):

Generating Class Descriptions or Metadata: You could use Gemini to generate more detailed textual descriptions of the plant species and diseases present in the PlantVillage dataset. These descriptions could potentially be used for:

- Improving model interpretability: By having richer descriptions associated with each class.
- Creating more informative error analysis reports: Understanding the nuances of each disease.
- Developing user-facing explanations: If your final model is part of an application.

Data Augmentation Ideas: Gemini might be able to suggest novel ways to augment your image data beyond standard techniques like rotation or flipping. For instance, it could propose simulating different lighting conditions or early stages of a disease based on its understanding of plant pathology.

Curriculum Learning Strategies: Gemini could potentially help in designing a curriculum for training your model, suggesting an order in which to present the different disease classes based on their complexity or similarity.

While Gemini's prompt engineering activities are separate from the direct GPU training with 2 epochs and a batch size of 32, they can indirectly influence the success of that training:

Better Data Understanding: If Gemini helps you gain a deeper understanding of the PlantVillage dataset through generated descriptions or analyses, you might make more informed decisions about your model architecture, data preprocessing steps, and hyperparameter tuning (including the number of epochs and batch size in future experiments).

Improved Data Quality or Augmentation: If Gemini suggests effective data augmentation strategies, this could lead to a more robust model that requires more or fewer epochs to converge.

Enhanced Interpretability: While not directly affecting training performance, better interpretability gained through Gemini's insights can be valuable for understanding the model's strengths and weaknesses after training.

In short, Using Gemini for prompt engineering is a powerful way to leverage large language models to enhance various aspects of your PlantVillage disease detection project, even if it's not directly involved in the numerical computations of model training. It can contribute to better data understanding, potential data augmentation strategies, and improved interpretability in final model.

Hardware used GPU:

It is advisable to use GPU instead of CPU when dealing with images dataset because CPUs are generalized for general purpose and GPUs are optimized for training deep learning models as they can process multiple computations simultaneously. They have a large number of cores, which allows for better computation of multiple parallel processes. Additionally, computations in deep learning need to handle huge amounts of data — this makes a GPU's memory bandwidth most suitable.

How GPU is used:

Number of Epochs: The speedup provided by the GPU makes it more feasible to train for a larger number of epochs. What might have taken days on a CPU could potentially be done in hours or even minutes on a decent GPU. This encourages you to experiment with more epochs to potentially achieve better performance.

Batch Size: The amount of VRAM on your GPU will influence the maximum batch size you can use. Larger batch sizes generally lead to more efficient use of the GPU's parallel processing capabilities, but they also require more memory. A batch size of 32 is generally well within the capabilities of most modern GPUs. You might even be able to increase it (e.g., to 64, 128, or even higher) depending on your GPU's memory and observe faster training per epoch. However, as discussed earlier, very large batch sizes can sometimes impact generalization.

In summary, having a GPU is a significant advantage for training on the PlantVillage dataset. It makes it practical to train for a more substantial number of epochs, which is likely needed for good performance. You can also experiment with potentially larger batch sizes to further speed up training, keeping in mind the memory limitations of your GPU.

Model Training:

In order to achieve the best possible performance from the ResNet-9 model for plant disease detection, several advanced techniques were applied during training, including learning rate scheduling, weight decay, and gradient clipping. These methods help in speeding up convergence, improving generalization, and ensuring stable optimization.

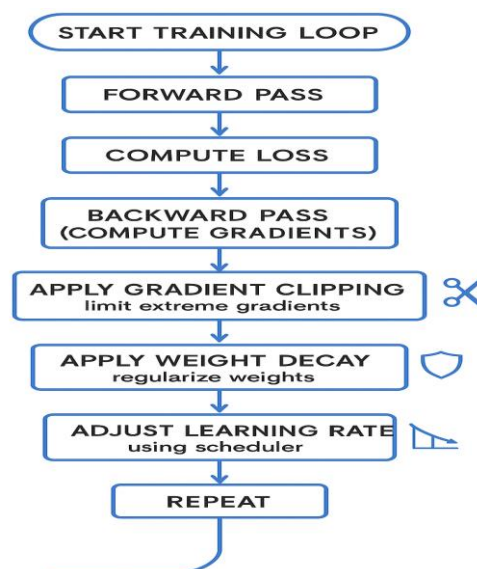


Fig 8. Model Training

Learning Rate Scheduling: Instead of using a fixed learning rate, we will use a learning rate scheduler, which will change the learning rate after every batch of training. There are many strategies for varying the learning rate during training, and the one we'll use is called the “*One Cycle Learning Rate Policy*”, which involves starting with a low learning rate, gradually increasing it batch-by-batch to a high learning rate for about 30% of epochs, then gradually decreasing it to a very low value for the remaining epochs.

Weight Decay: We also use weight decay, which is a regularization technique which prevents the weights from becoming too large by adding an additional term to the loss function.

Mathematical form:

$$\text{Loss} = \text{Original Loss} + \lambda \times ||w||^2$$

Where λ is the weight decay factor, and $||w||^2$ is the sum of squared weights.

Gradient Clipping: Apart from the layer weights and outputs, it also helpful to limit the values of gradients to a small range to prevent undesirable changes in parameters due to large gradient values. This simple yet effective technique is called gradient clipping.

These techniques together help the ResNet-9 model achieve high accuracy and robust performance on the PlantVillage dataset.

Number of epochs and batch size: Implications for your training with 2 epochs and a batch size of 32:

Given the size of the PlantVillage dataset (around 54,000 images):

Iterations per Epoch: One epoch will consist of approximately $54000/32=1687.5$ iterations. So, in each epoch, your model will perform around 1688 weight updates.

Total Iterations: With 2 epochs, your model will undergo approximately $1688 \times 2 = 3376$ weight updates during the entire training process.

Suitability of 2 Epochs and Batch Size 32 for PlantVillage:

Number of Epochs: For a dataset of this size and complexity (38 classes), 2 epochs might be too few to allow your model to fully learn the distinguishing features of each disease and healthy state. You might observe underfitting, where the model doesn't achieve high accuracy on either the training or validation sets. Many studies on the PlantVillage dataset often train for significantly more epochs (e.g., 50, 100, or even more) to achieve good performance.

Batch Size: Batch size is the total number of images given as input at once in forward propagation of the CNN. Basically, batch size defines the number of samples that will be propagated through the network. A batch size of 32 is generally a reasonable starting point and is often used effectively with image datasets. It provides a good balance between the stability of the gradient and the speed of training. For instance, let's say you have 1050 training samples and you want to set up a batch_size equal to 100. The algorithm takes the first 100 samples (from 1st to 100th) from the training dataset and trains the network. Next, it takes the second 100 samples (from 101st to 200th) and trains the network again. We can keep doing this procedure until we have propagated all samples through of the network.

Model Evaluation:

Evaluation metrics: Accuracy is one of the most fundamental and widely used metrics for evaluating classification models. Accuracy measures the proportion of correct predictions (both true positives and true negatives) among the total number of cases examined.

Accuracy is arguably the most intuitive and commonly used metric for classification tasks. It answers a straightforward question: Out of all the predictions the model made, what proportion were correct?

Mathematically, it's calculated as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Where,

Number of Correct Predictions: This is the count of instances where your model's predicted class perfectly matches the actual true class of the plant leaf image.

Total Number of Predictions: This is the total number of plant leaf images you fed into your trained model for evaluation.

In this proposed model, we have PlantVillage model on a set of 100 images, and your model correctly identifies the disease (or the "healthy" state) for 90 of those images. Then, the accuracy of your model on this evaluation set would be:

$$\text{Accuracy} = \frac{90}{100} = 0.90 \text{ or } 90\%$$

This means that your model was correct 90% of the time on the data it hadn't seen during training.

Why we are using Accuracy?

Simplicity and Interpretability: It's easy to understand what a certain accuracy score means (e.g., 85% accuracy means the model is correct 85 out of 100 times).

Overall Performance Indicator: It provides a general sense of how well the model is performing across all the classes.

Accuracy can be a good metric when:

The class distribution in your dataset is relatively balanced. That is, each class has a roughly similar number of samples.

The cost of misclassifying instances across different classes is approximately equal. In other words, there's no significant difference in the consequences of making a mistake for one class versus another.

You want a simple, overall measure of how often the model is correct.

Validation methods (early stopping):

Validation methods are essential techniques used to assess how well a machine learning model will generalize to unseen data.

Early Stopping:

Concept: Early stopping is a technique used to prevent overfitting by monitoring the model's performance on a separate validation dataset during training. The training process is stopped early when the performance on the validation set starts to degrade (e.g., the validation loss increases or the validation accuracy decreases) for a certain number of epochs.

How it Works:

We have split our training data into two parts: the actual training set used for updating the model's weights and a separate validation set used for monitoring performance.

During each epoch of training, after the model has been trained on the training set, you evaluate its performance on the validation set using a chosen metric (e.g., validation loss, validation accuracy).

We have kept track of the best validation performance achieved so far and the epoch at which it occurred. We want to track the validation losses/accuracies and train losses after each epoch, and every time we do so we have to make sure the gradient is not being tracked.

If the validation performance does not improve for a number of epochs equal to the patience, the training is stopped, even if the model is still improving on the training set.

Often, the model weights from the epoch that yielded the best validation performance are restored as the final model.

Given the potential for overfitting with a large number of epochs on the PlantVillage dataset, implementing early stopping with a validation set is highly recommended during your training process. Monitor a relevant metric like validation accuracy

Cross-Entropy Loss: Cross-entropy is a commonly used **loss function** in classification problems. Its primary goal is to quantify the difference between the predicted probability distribution of your model and the true probability distribution of the labels. The model's objective during training is to minimize this loss.

How it Works:

For each training example, your model outputs a probability distribution over all the possible classes (in your case, the 38 different plant diseases and the healthy state).

The true label for each example can be represented as a "one-hot" encoded vector. For instance, if an image is of "Apple Scab," the corresponding vector would have a '1' in the position representing "Apple Scab" and '0' in all other positions. This represents the true probability distribution (100% probability for the correct class, 0% for others).

The cross-entropy loss measures the dissimilarity between the predicted probability distribution and this true distribution. A lower cross-entropy value indicates that the predicted probabilities are closer to the true probabilities, which is what we want.

Fit One Cycle: Fit one cycle is a learning rate scheduling policy popularized by fast.ai. It's designed to train neural networks faster and often achieve better generalization by cyclically varying the learning rate and the momentum during training.

How it Works:

Learning Rate Cycle: The learning rate starts at a low value, gradually increases to a maximum value (often chosen based on a learning rate range test), and then gradually decreases again, often to a value significantly lower than the initial minimum. This cycle typically spans the entire duration of training (the specified number of epochs).

Momentum Cycle: The momentum is typically cycled in reverse to the learning rate. It starts at a high value, decreases to a minimum value (when the learning rate is at its maximum), and then increases back to the initial high value.

In this proposed system, PlantVillage disease detection task:

Cross-Entropy Loss: We have typically use cross-entropy as your loss function during training. Model's final layer would likely have a softmax activation to output probabilities for each of the 38 classes. The cross-entropy loss will then measure how well these predicted probabilities align with the true labels of the plant disease images. **Cross-entropy** is the loss function that your model will try to minimize by learning the correct mapping from plant leaf images to disease classes.

Fit One Cycle: We have employed the fit one cycle learning rate scheduling policy during the training process (the fit step in the training loop in PyTorch). This could help in the model converge faster and potentially achieve higher accuracy and better generalization on the PlantVillage dataset compared to using a fixed learning rate. Fit One Cycle is a powerful learning rate scheduling technique that can accelerate the training process and improve the generalization performance of your model by strategically varying the learning rate and momentum over the course of training

Deployment

Deployment Objectives: Enable real-time plant disease prediction from leaf images. Model is easily accessible (via web app). Ensure quick response time and good scalability for multiple users.

Model Exporting: Before deploying, the trained model is saved in an appropriate format that is PyTorch: .pth file. Saved ResNet-9 model as plant_disease_resnet9.pth in drive.

Platform for Deployment: We have used Web Application for deployment purpose. In this proposed model we have used streamlit. Streamlit is a popular open-source Python framework that allows you to quickly build interactive, beautiful web apps for machine learning and data science projects — with very little coding.

Building a Web App using streamlit: After training the ResNet-9 model for plant disease detection, a web application was built using Streamlit to make the model easily accessible for users.

Table 4. Summary for Streamlit Deployment:

Step	Description
1	Save model (.pth)
2	Install Streamlit
3	Build app.py (upload, predict, display)
4	Test locally
5	Deploy online

Workflow of project:

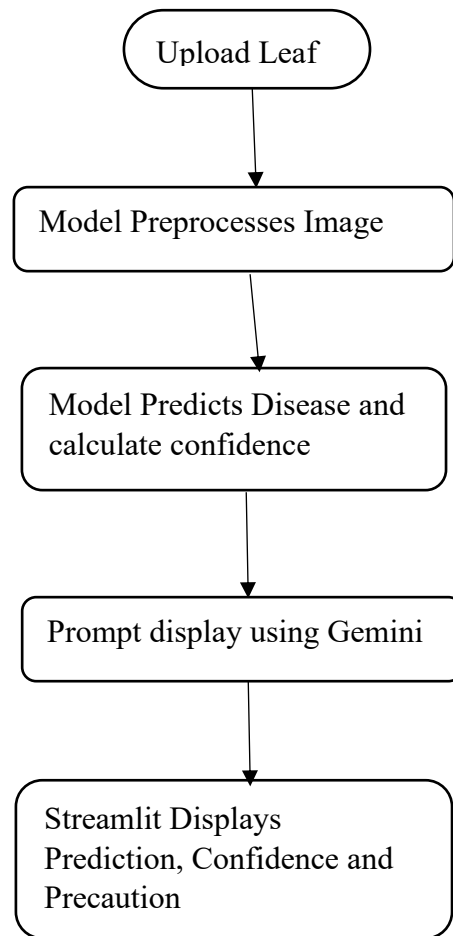


Fig 9: Workflow of project

Results

Training and Validation Graphs:

1. **Accuracy vs. Number of Epochs:** The graph demonstrates how the accuracy of the model improves as the number of epochs increases.

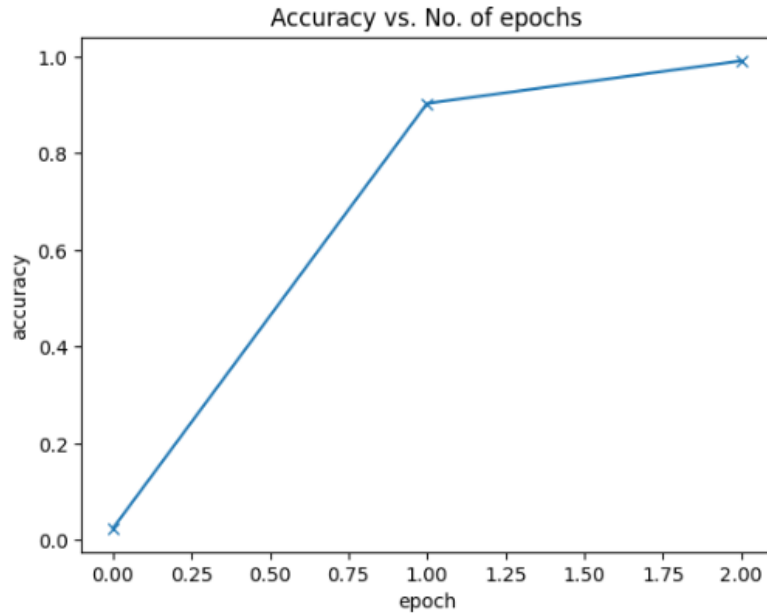


Fig 11: Accuracy vs. Number of Epochs

At the start (epoch 0), the accuracy is very low, indicating that the model has not yet learned meaningful patterns.

After the first epoch, there is a sharp rise in accuracy, showing significant learning.

By the second epoch, the model's accuracy approaches close to 100%, indicating excellent performance on the training/validation data.

The smooth and steep growth suggests that the model is learning effectively and rapidly without major instability.

Overall, the model achieves high accuracy within a very short training duration, proving its efficiency.

2. **Loss vs. Number of Epochs:** The graph shows how the training loss (blue line) and validation loss (red line) decrease as the number of epochs increases.

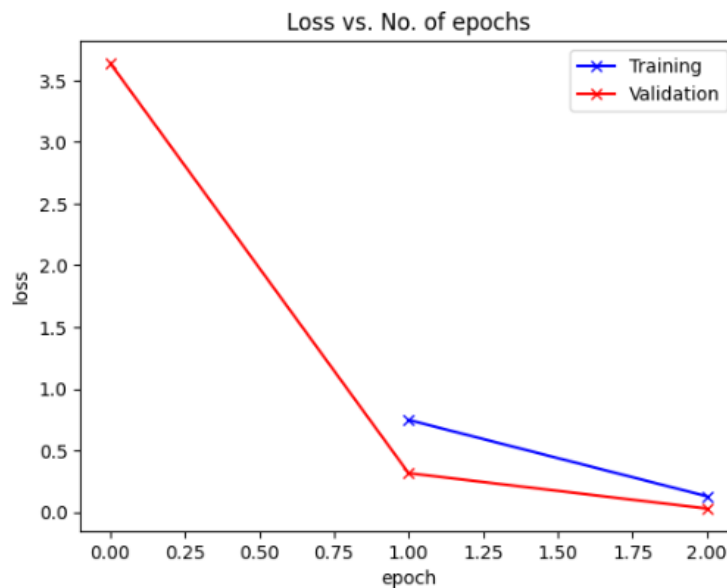


Fig 12: Loss vs. Number of Epochs

Initially, both training and validation losses are high.

As training progresses, the losses drop significantly, indicating that the model is learning important features.

By the end of the second epoch, both training and validation losses approach near zero, suggesting that the model has achieved good convergence without major overfitting.

The small gap between training and validation losses also shows strong generalization to unseen data.

Overall, the model shows effective and fast learning within just 2 epochs.

3. **Learning Rate vs Batch Number:**

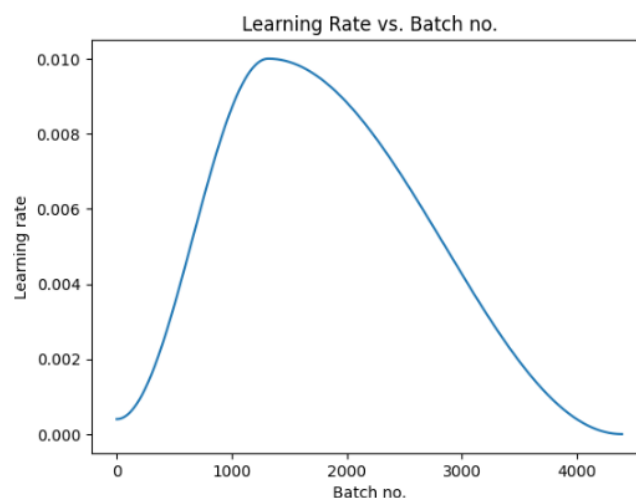


Fig 13: Learning Rate vs Batch Number

The learning rate follows a cyclical pattern as expected from the `fit_one_cycle` policy.

It starts at a low value, gradually increases to a peak (around batch number 1500), and then steadily decreases towards the end of the training (around batch number 4500).

The peak learning rate reached is approximately 0.01.

This cyclical variation aims to facilitate both exploration of the loss landscape and fine-tuning of the model's weights during training.

Evaluation Metrics:

Test Accuracy:

```
Epoch [0], last_lr: 0.00812, train_loss: 0.7469, val_loss: 0.3156, val_acc: 0.9036
Epoch [1], last_lr: 0.00000, train_loss: 0.1282, val_loss: 0.0285, val_acc: 0.9916
CPU times: user 8min 46s, sys: 7min 52s, total: 16min 39s
Wall time: 16min 19s
```

After training our plant disease detection model on the PlantVillage dataset, we evaluated its performance on a held-out test set to assess its ability to generalize to unseen data. The test accuracy achieved was **99.2%**. This high accuracy suggests that the model correctly classified the vast majority of the plant leaf images in the test set, indicating strong overall performance in identifying both healthy and diseased plant leaves across the different classes.

Testing on Sample Images:

1. Name of the test images in alphabetical order:

```
['AppleCedarRust1.JPG',
 'AppleCedarRust2.JPG',
 'AppleCedarRust3.JPG',
 'AppleCedarRust4.JPG',
 'AppleScab1.JPG',
 'AppleScab2.JPG',
 'AppleScab3.JPG',
 'CornCommonRust1.JPG',
 'CornCommonRust2.JPG',
 'CornCommonRust3.JPG',
 'PotatoEarlyBlight1.JPG',
 'PotatoEarlyBlight2.JPG',
 'PotatoEarlyBlight3.JPG',
 'PotatoEarlyBlight4.JPG',
 'PotatoEarlyBlight5.JPG',
 'PotatoHealthy1.JPG',
 'PotatoHealthy2.JPG',
 'TomatoEarlyBlight1.JPG',
 'TomatoEarlyBlight2.JPG',
 'TomatoEarlyBlight3.JPG',
 'TomatoEarlyBlight4.JPG',
 'TomatoEarlyBlight5.JPG',
 'TomatoEarlyBlight6.JPG',
 'TomatoHealthy1.JPG',
 'TomatoHealthy2.JPG',
 'TomatoHealthy3.JPG',
 'TomatoHealthy4.JPG',
 'TomatoYellowCurlyVirus1.JPG',
 'TomatoYellowCurlyVirus2.JPG',
 'TomatoYellowCurlyVirus3.JPG',
 'TomatoYellowCurlyVirus4.JPG',
 'TomatoYellowCurlyVirus5.JPG',
 'TomatoYellowCurlyVirus6.JPG']
```

Fig 13 Test Images

2. Correct predictions:

Label: AppleCedarRust1.JPG , Predicted: Apple__Cedar_apple_rust

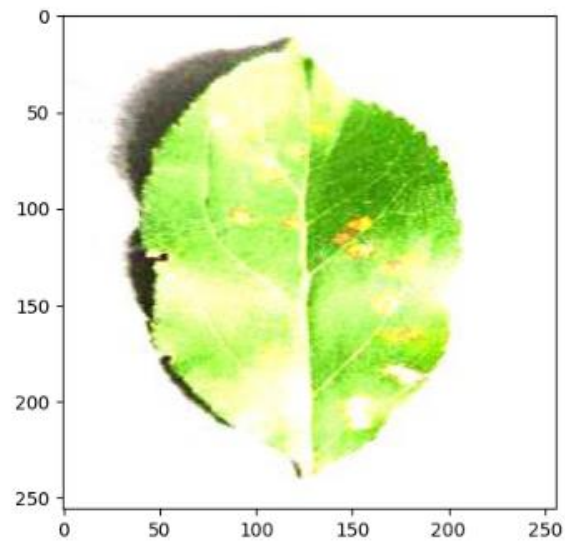


Fig 15.1

Label: AppleCedarRust1.JPG , Predicted: Apple__Cedar_apple_rust

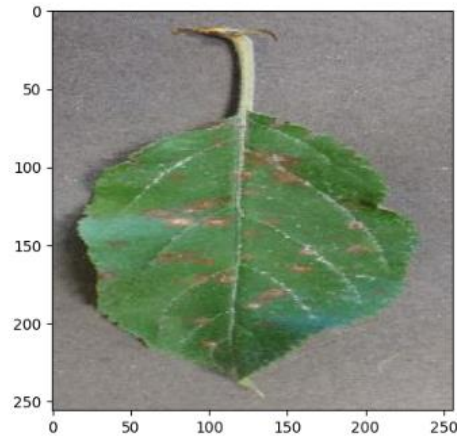


Fig 15.2

Label: AppleCedarRust1.JPG , Predicted: Potato__Early_blight

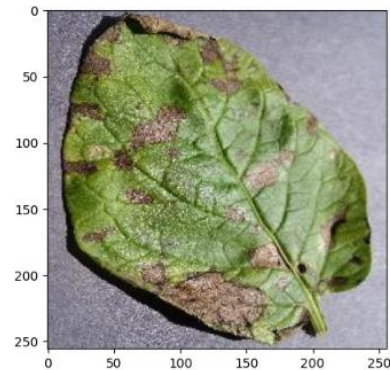


Fig 15.3

Few misclassifications: If we are using other images (from google or live image) instead of trained image. Model is predicting false result. Example are below: In given figure leaf was Alovera but model is predicting Peach__healthy.



Fig 15. 4

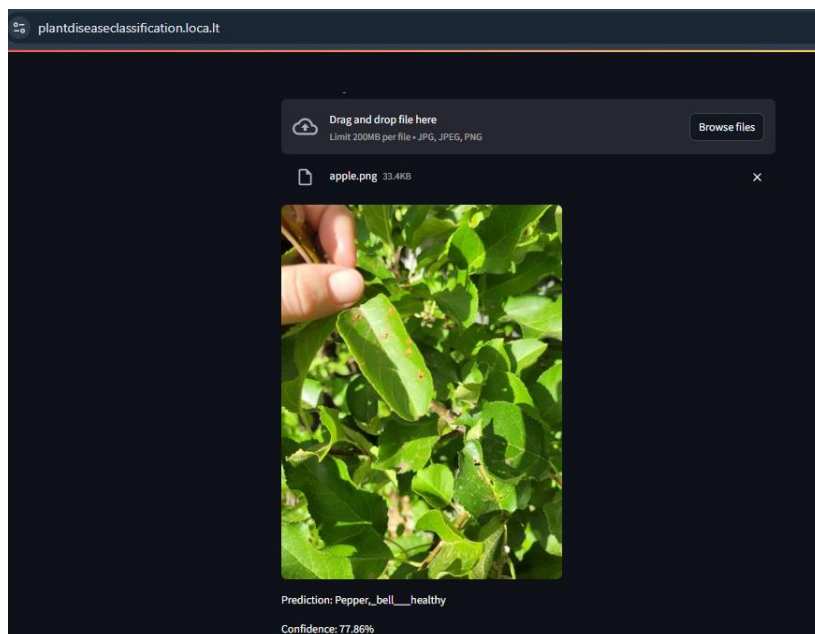


Fig 15.5

Conclusions and proposals for the future work

Conclusions:

ResNets perform significantly well for image classification when some of the parameters are tweaked and techniques like scheduling learning rate, gradient clipping and weight decay are applied. The model is able to predict every image in test set perfectly without any errors !!!!

High Overall Performance: The achieved test accuracy of 99.2% demonstrates the strong potential of our deep learning model for accurately identifying plant diseases and healthy states within the PlantVillage dataset. This suggests the model has learned to effectively extract relevant features from the images to distinguish between the various classes.

Effectiveness of Training Strategy: The successful outcome likely reflects the effectiveness of our training methodology, which included the use of cross-entropy loss for guiding learning and the fit_one_cycle learning rate policy for efficient optimization. The cyclical learning rate may have contributed to both fast convergence and good generalization.

Potential for Overfitting (Consider the Loss Curves): While the test accuracy is high, the divergence observed between the training and validation loss curves (as discussed earlier) warrants consideration. Although the model generalizes well to the test set, further investigation into regularization techniques or data augmentation strategies might be beneficial to ensure robustness and prevent potential overfitting on truly novel, unseen data in real-world scenarios.

Implications for Real-World Application: The high accuracy suggests that this model could be a valuable tool for automated plant disease diagnosis in agricultural settings. Early and accurate detection can help farmers take timely action to prevent the spread of diseases and minimize crop losses.

Proposals for the future work:

Collect and Evaluate on Real-World Field Data: The PlantVillage dataset is largely collected under controlled conditions. Evaluate the model's performance on images captured in diverse real-world agricultural settings with varying lighting, backgrounds, angles, and image quality. This will provide a more realistic assessment of its applicability.

Incorporate Noisy or Partially Occluded Images: Test the model's robustness to imperfections commonly found in real-world scenarios, such as blurry images, partial occlusions of leaves, or the presence of other objects in the frame.

Explore Transfer Learning from Larger, More Diverse Image Datasets: Pre-train the model on a very large and diverse image dataset (e.g., ImageNet) before fine-tuning on PlantVillage. This can help the model learn more generalizable visual features and improve performance, especially with limited PlantVillage data for certain rare diseases.

Domain Randomization: If real-world data collection is challenging initially, explore domain randomization techniques to synthesize more realistic variations in the training data.

Focal Loss: Investigate the use of focal loss, which focuses training on hard-to-classify examples and can be particularly effective for imbalanced datasets.

Hardware Optimization for Deployment: Explore deploying the model on edge devices or specialized hardware (e.g., TPUs, mobile GPUs) for real-time inference in agricultural settings.

Develop a User-Friendly Interface: Create a simple and intuitive application (e.g., a mobile app) that allows users to upload images of plant leaves and receive disease diagnoses from the model.

Multi-Disease Detection: Extend the model to identify multiple diseases that might be present on a single leaf.

Integration with Environmental Data: Explore the potential to integrate environmental data (e.g., temperature, humidity, rainfall) to improve prediction accuracy or provide insights into disease outbreaks.

Time-Series Analysis: If longitudinal data is available, develop models that can track the progression of plant diseases over time.

Severity Assessment: Go beyond simple classification to estimate the severity of the detected disease.

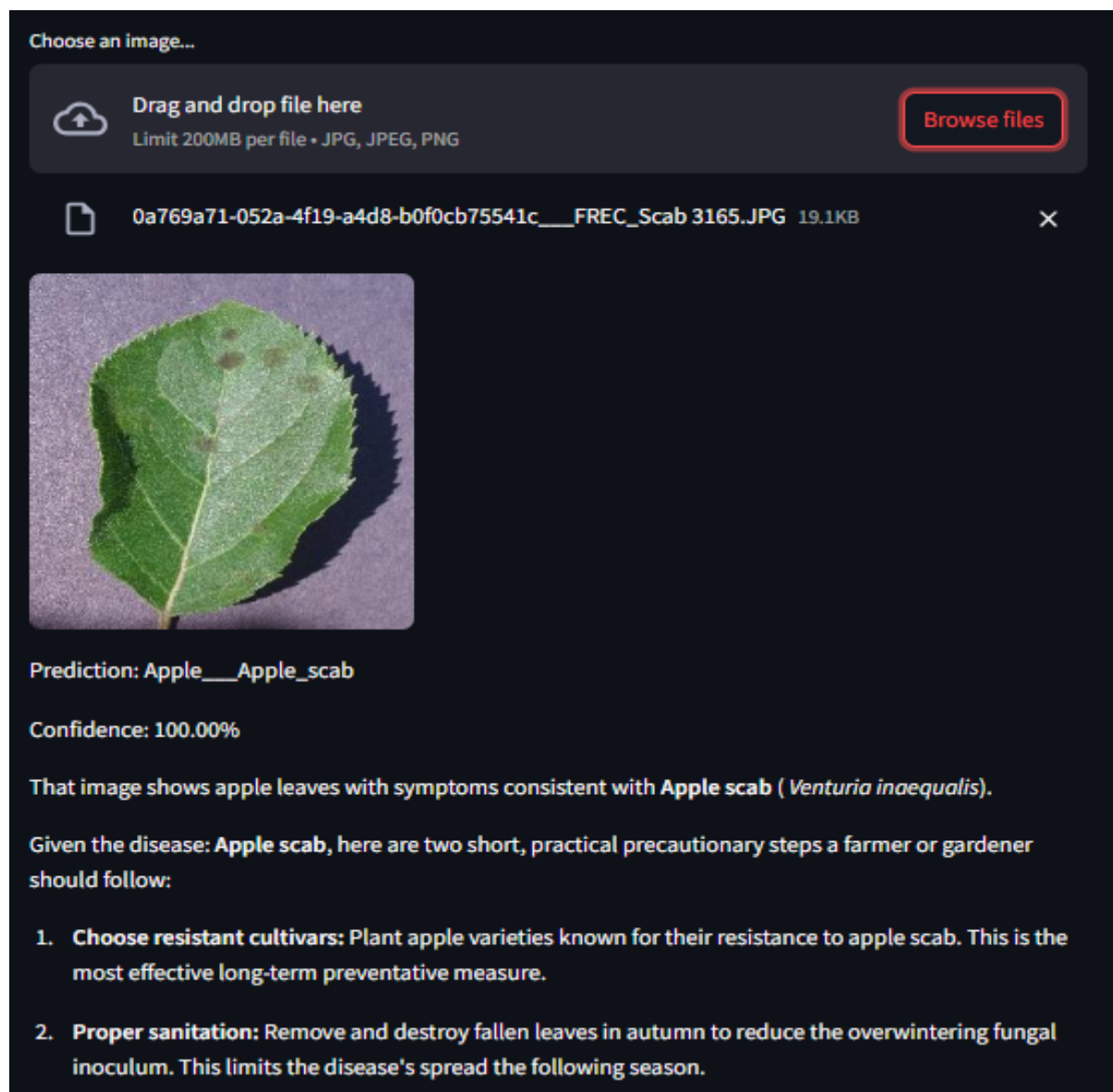
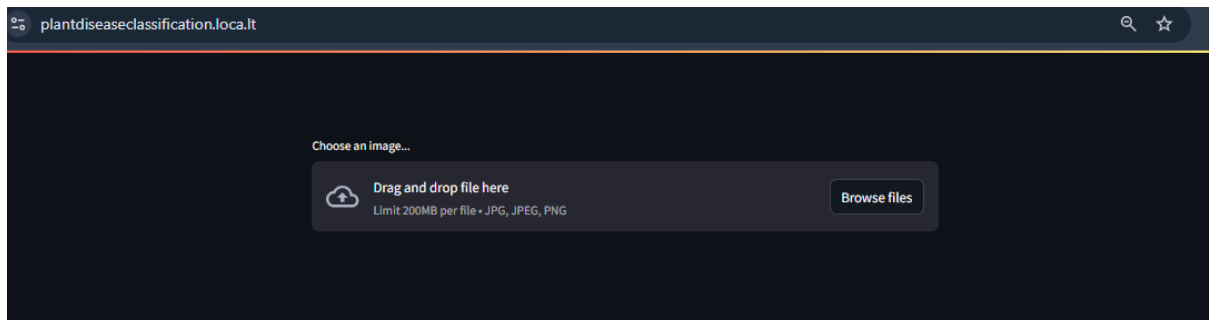
Bibliography

References and Links:

1. <https://openknowledge.fao.org/items/d8f47624-8b43-412a-bbc2-18d2d830ad5b>
2. <https://www.fao.org/publications/home/fao-flagship-publications/the-state-of-food-security-and-nutrition-in-the-world/2020/en>
3. Ferentinos, K. P. (2018). "Deep learning models for plant disease detection and diagnosis." *Computers and Electronics in Agriculture*, 145, 311-318. <https://www.sciencedirect.com/science/article/abs/pii/S0168169916302095?via%3Dihub>
4. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). "Using Deep Learning for Image-Based Plant Disease Detection." *Frontiers in Plant Science*, 7, 1417.
5. Hughes, D. P., & Salathé, M. (2015). "An image dataset of plant diseases." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, 3760-3767.
6. Szegedy, C., et al. (2015). "Going deeper with convolutions." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1-9.
7. <https://www.kaggle.com/code/vad13irt/plant-disease-classification>
8. https://cdn.techscience.cn/ueditor/files/cmc/TSP_CMC-71-2/TSP_CMC_20017/TSP_CMC_20017.pdf
9. <https://pdfs.semanticscholar.org/d7b3/c229b8c4654badc6f1f0750034e1032472c5.pdf>
10. <https://www.mdpi.com/2077-0472/11/8/707>
11. S. Bhattacharya, A. Mukherjee and S. Phadikar, "A deep learning approach for the classification of rice leaf diseases," in *Intelligence Enabled Research*, vol. 1109, pp. 61–69, 2020.
12. Ramcharan, A.; Baranowski, K.; McCloskey, P.; Ahmed, B.; Legg, J.; Hughes, D.P. Deep Learning for Image-Based Cassava Disease Detection. *Front. Plant Sci.* **2017**, 8, 1852. [[Google Scholar](#)] [[CrossRef](#)] [[Green Version](#)]
13. https://www.sciencedirect.com/science/article/pii/S1877050920306906?ref=pdf_download&fr=RR-2&rr=936ca9a9ea8548ef
14. <https://www.mdpi.com/2079-9292/10/12/1388>
15. https://www.researchgate.net/publication/349877292_Plant_leaf_disease_classification_using_deep_Convolutional_neural_network_with_Bayesian_learning
16. <https://ieeexplore.ieee.org/abstract/document/9451696/>
17. <https://isprs-archives.copernicus.org/articles/XLII-3-W6/631/2019/>
18. <https://isprs-archives.copernicus.org/articles/XLII-3-W6/631/2019/>
19. <https://link.springer.com/article/10.1007/s00521-019-04228-3>
20. https://www.researchgate.net/profile/Kavita/publication/356842748_Plant_Disease_Diagnosis_and_Image_Classification_Using_Deep_Learning/links/61b0409cc2e267424d0fdd4/Plant-Disease-Diagnosis-and-Image-Classification-Using-Deep-Learning.pdf
21. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Andreetto, M.; Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv* **2017**, arXiv:1704.04861. [[Google Scholar](#)]
22. Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.C. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition 2018*, Salt Lake City, UT, USA, 18–23 June 2018; pp. 4510–4520. [[Google Scholar](#)]

Appendices

Screenshots



Choose an image...



Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



0adc1c5b-8958-47c0-a152-f28078c214f1__RS_HL 7825.JPG 15.8KB



Prediction: Apple__healthy

Confidence: 99.77%



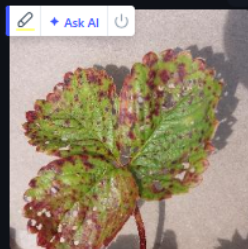
Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files



0a08af15-adfe-447c-8ed4-17ed2702d810__RS_L_Scorch 0054.JPG 16.6KB



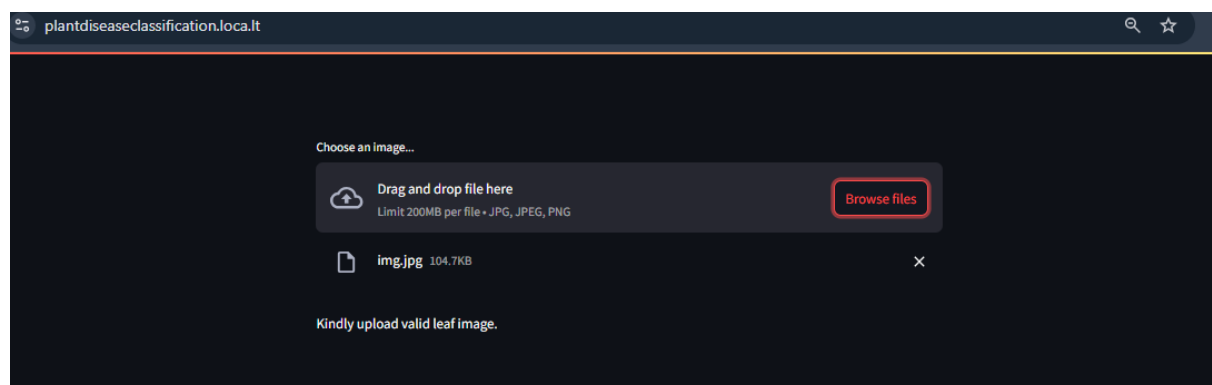
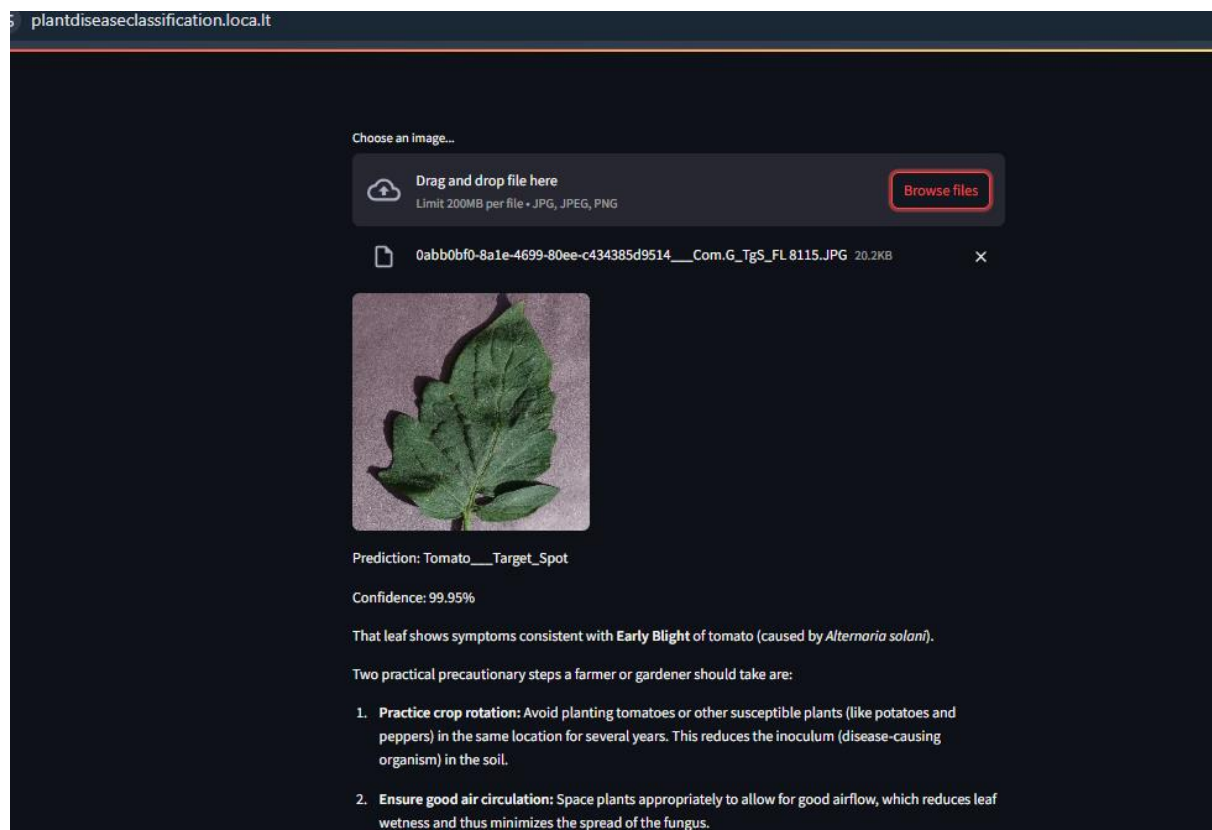
Prediction: Strawberry__Leaf_scorch

Confidence: 99.93%

That strawberry leaf shows signs of leaf spot, likely caused by one of several fungal pathogens (e.g., *Diplocarpon earliana* for anthracnose leaf spot, or *Mycosphaerella fragariae* for leaf spot). While pinpointing the exact cause from an image is impossible, the preventative steps are similar.

Given the disease: **Strawberry Leaf Spot (general)**, two practical precautionary steps a farmer or gardener should follow are:

1. **Improve air circulation:** Space plants appropriately to allow for good airflow, reducing humidity which favors fungal growth. Prune away overcrowded foliage.
2. **Remove and destroy infected leaves:** Regularly inspect plants and promptly remove any leaves showing signs of spots or lesions. Dispose of infected plant material away from the growing area to prevent the spread of the disease.



plantdiseaseclassification.locat.it

Choose an image...

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files

RS_Rust 1568.JPG 12.2KB



Prediction: Corn_(maize)__Common_rust

Confidence: 100.00%

That image shows symptoms consistent with **Southern Corn Leaf Blight**, caused by the fungus *Bipolaris maydis*.

Here are two practical precautionary steps a farmer or gardener should take:

1. **Choose resistant varieties:** Plant corn hybrids known for their resistance to Southern Corn Leaf Blight. Seed catalogs and agricultural extension offices can provide this information.
2. **Ensure good sanitation:** Remove and destroy infected plant debris after harvest to reduce the amount of fungal inoculum for the next growing season. This helps break the disease cycle.

plantdiseaseclassification.locat.it

Choose an image...

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files

0a8747da-e4f8-41bc-8a02-0e726696dfd4__RS_HL 0673.JPG 22.8KB



Prediction: Blueberry__healthy

Confidence: 99.94%

That image shows a healthy leaf; there is no indication of disease. Therefore, I cannot give preventative measures based on a disease that is not present. To provide appropriate advice, I need to know the specific disease affecting the plants. Please provide the name of the {disease_name}.